L1 - Intro to Python

- computer → compute
  how do computers compute?
  → data movement → read / store
  → logical → compare / and / or
  → operations → +, -, x, ÷, ++, +-
  → control flow → jump to another instruction.
- Programming is a process of organising basic operations into simple applications which combine to solve a much larger problem.
- Program is a seq. of def^n (evaluated) and commands (executed). They can be typed directly in the shell. Or they can be stored in a file.
- syntactic errors → common, easily caught by compiler.
- Programs manipulate data objects. Objects have a type.
- Objects are scaler (can't be divided) or non-scaler (have internal struc. that can be accessed).
- Scaler obj. → int, float, bool, NoneType
- Type conversion (known as casting) can convert obj. of 1 type to another. float (3), int (3.9).
- Variables can be used to modify the code by modifying basic components.
- round (). for rounding nos. to some no of decimal places. round (3.1415, 2)
  abs () → gets the absolute value of the no.
  pow () → raises a no to some power.
  pow (2, 3), or pow (2, 3, 2)
  ↓
  mod 2

- String – collection of text, we can concatenate strings, we have string indexing that starts with 0.
- A portion of a string is called substring. It can be achieved using string slicing.
- Strings are immutable, they can't be changed once we have made them.
- we can user input → input ().
  the result is ← always string.
- Comments are used to clarify code that may not be easy to understand. It explains why sth. is done in a certain way.
- Functions are building blocks of every python prog. There are some built-in functions like print(), len(), etc. They break the code into smaller chunks and can be re-used.
- In Python, functions are values & can be assigned to a variable.
- The del keyword (del(un)) is to un-assign a variable from its value.
- You must call the function explicitly in Python to execute it.
- An argument is a value that gets passed to the funcn as an input. Funcn can have no arguments. Once funcn is completed, it returns a value.
- Every funcn has 2 parts – function sign. that defines name of the funcn & any input it expects, and funcn body that contains the code that runs everytime the funcn is c/d.
- Indentan is impt. in Python.

- All func^n in Python return a value, even if that value is None. However, not all func^n need a return statement.
- help(fun) ⇒ tells what your func^n does. For custom func^n,

```
def multiply(x, y):
    """
    _______________
    """
    func^n body ___
```

this part will appear in help(multiply)

- A loop is a block of code that gets repeated over & over until some condition is met (while) or no of specified iterations (for).
- In while loop, if you aren't careful, you might create an infinite loop.
- When is while loop appropriate → if the no. of terms reqd. is not known in advance or the computat^n should stop only when a desired accuracy is achieved.
-

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{3}{3!}$$

do this until $\frac{1}{n!} < 10^{-8}$

```
n = 0, term = 1, total = 0
while ter > 1e-8:
    total = total + term
    n = n + 1
    term = term / n
```

- root using while loop

$f(x)$:
$x^3 - x - 2$

```
a = 1, b = 2    tol = 1e-6
while (b - a) > tol:
        c = (a + b) / 2
        if f(a) x f(c) < 0:
            b = c
        else: a = c
```

root $= \frac{a+b}{2}$

- A for loop executes a secⁿ of code for each item in a collecⁿ of items. The no. of times that code is executed is determined by the no. of collecⁿ items.
-
$$I = \int_0^1 x^2 \, dx \qquad \Delta x = \frac{1}{N}$$

$$I = \sum_{x=0}^{N-1} f(x_i) \Delta x \, , \quad x_i = i \cdot \Delta x \, , \quad f(x) = x^2$$

```
f(x):
    x^2

N = 100,   dx = 1/N     inte = 0
for i in range(100):
    x = i . dx
    inte = inte + f(x) . dx
```

- We can have nested loops (loop within loop).
- Scopes follow a heiraarchial system. The funcⁿ body has what is known as the local scope. Code outside the funcⁿ body has global scope.

  L → E → G → B
  (local) (enclosing) (global) (built-in)
- Programs often need to make decisions based on certain conditions.
  if → elif → else
- Strings can be compared just like nos.
  ==, !=, <, >, <=, >=, in, not in
  → uppercase & lowercase letters are diff.
  → comparison starts from the first charac.
- The in operator checks whether a substring is present in a string. Remember, string matching is case-sensitive.

- Boolean expressions → and, or, not

why is this operator useful

→ It helps to negate an entire condition w/o having to rewrite the entire logic.

→ if the conditions are compound, this make the code easier to read & less error-prone.